Non-Functional Requirements (NFR) Benchmarking for IT Automation Agents

Arjun Vaidya, Ayush Bhauwala, Gautam Agarwal, Rishabh Jain, Tanmay Agrawal
Department of Computer Science, *Columbia University*, New York, USA
{av3315, ab6106, ga2726, rj2790, ta2832}@columbia.edu

***Abstract*—AI agents are increasingly being deployed in IT automation domains such as SRE, CISO operations, and FinOps, yet their evaluation remains dominated by functional success metrics that overlook practical deployment and performance risks. Agentic systems exhibit non-deterministic reasoning, multi-step tool use, compounding errors and brittle recovery mechanisms, and sometimes succeed through unintended paths, revealing a critical gap between benchmark performance and production readiness. We first synthesize findings from software engineering, ML system quality models, and recent agentic frameworks to establish a unified understanding of non-functional requirements (NFRs) for AI agents. We extend ITBench with an NFR-aware evaluation framework, providing a comprehensive insights into NFR characteristics of ITBench SRE and CISO Agents. Our code is publicly available at github.com/ITBench-NFR**

***Index Terms*—AI agents, non-functional requirements, benchmarking, IT automation, observability, production readiness, performance metrics**

I. Introduction

Recent progress in LLMs has accelerated the development of AI agents capable of interacting with tools within complex IT environments. Current AI agent benchmarks such as IT-Bench [7], AgentBench [9], and others predominantly evaluate functional capabilities through task completion rates, revealing significant performance gaps in production scenarios.

While functional metrics reveal important capability gaps, they provide limited insight into whether agents can operate reliably, efficiently, and safely in production settings. Results from current benchmarks illustrate this limitation, agents may succeed functionally yet exhibit excessive latency, high token and API costs, brittle recovery behavior, or insufficient execution traceability. In production IT systems, such deficiencies can negate the value of functional success, exposing a growing gap between benchmark performance and real-world deployment viability.

A. Background and Motivation

AI agents for IT automation employ diverse architectural patterns, ReAct (Reasoning and Acting), Plan&Execute, and Reflection-based approaches, each creating distinct trade-offs.

ReAct agents interleave reasoning and action steps, potentially achieving lower latency but higher token consumption through iterative tool calls. Plan&Execute architectures separate planning from execution, offering more reliable workflows at the cost of increased planning overhead. These architectural choices directly impact the four critical NFR dimensions: performance (end-to-end latency and throughput), cost efficiency (token consumption and API call frequency), reliability (error rates and graceful degradation), and observability (trace completeness and decision explainability).

Moreover, NFR priorities are inherently domain-specific. IT automation spans three distinct operational domains with varying NFR criticality. In SRE scenarios, latency requirements are stringent, incident response delays measured in seconds can translate to millions in downtime costs, making time-to-first-token and planning overhead paramount. CISO operations prioritize reliability and auditability, where false positives in threat detection or incomplete compliance traces create regulatory risk. FinOps automation demands accuracy in financial calculations and cost efficiency paradoxically requiring that agent operational expenses be justified by savings generated. Current benchmarks measure domain-specific task success but neglect these contextual NFR thresholds that determine production viability.

B. Problem Statement

Non-Functional Requirements (NFRs), the quality attributes that define how a system operates rather than what it does, remain largely unmeasured in existing agent evaluation frameworks. This gap is critical because systems can satisfy functional requirements yet fail catastrophically on operational metrics such as latency during incident response, cost efficiency at scale, or observability for debugging failures. As a result, the distinction between laboratory success and production readiness hinges on comprehensive NFR evaluation across performance, cost, reliability, and observability dimensions.

C. Objectives and Scope

This project identifies the non-functional requirements (NFRs) that are critical for deploying production-ready AI agents in real-world IT environments. We extend ITBench by incorporating NFR-aware evaluation for SRE and CISO agents, and present a comprehensive taxonomy of agent-specific NFRs tailored to agentic workflows. We define concrete metrics for these NFRs, implement the necessary instrumentation to measure them, and evaluate SRE, CISO, and Mini-SWE agents using these metrics. Through this evaluation, we identify research opportunities for bridging the gap between benchmark performance and production readiness.

II. Literature Review

This section reviews the evolution of NFR frameworks, starting with traditional software engineering standards, examining their adaptation for ML systems, and analyzing the emerging, yet fragmented, landscape of NFRs for Agentic AI.

The foundational standard for NFR metrics is ISO/IEC 25010 [4], which establishes a software quality model comprising eight broad categories for describing software behavior. A comprehensive review of 182 sources over the last three decades identifies the five most frequently cited NFR types as Performance (88.68%), Reliability (67.92%), Usability (62.26%), Security (60.38%), and Maintainability (54.72%) [10].

With the rise of ML systems, traditional NFR frameworks have proven insufficient, prompting the adoption of new standards such as ISO/IEC 5338 [6] for the AI system life cycle and ISO/IEC 25059 [5], which specifically defines a quality model for AI systems. Building on these foundations, research extending the ISO 25010 quality model has explicitly addressed the gap by defining 34 quality characteristics, 18 of which, such as understandability, training data suitability, and robustness against noisy data, are unique to ML systems [11].

Practitioner perspectives reinforce these theoretical models. Surveys of industry and academic experts indicate a consensus that while NFRs remain essential for quality, they require different definitions and measurement approaches compared to traditional software, particularly regarding adaptability and maintainability. While foundational metrics like accuracy, reliability, integrity, and security remain paramount, the ML context has elevated the importance of qualities such as fairness, transparency, and explainability [3]. Further bibliometric analysis has identified 31 distinct NFRs for ML systems, with accuracy and robustness appearing most frequently, followed by fairness, security, and performance. Crucially, this research highlights that NFR prioritization is heavily domain-dependent, meaning the importance of specific NFRs shifts based on the deployment environment [1].

While NFRs for general software systems are well established, specific research regarding NFRs for AI Agents remains limited compared to general AI/ML systems. Addressing this gap, recent research has introduced frameworks such as "AgentOps" to establish observability and safety specifically for agentic systems. This approach identifies that, unlike traditional software, agents require the tracing of unique artifacts, specifically reasoning, planning, workflows, and guardrails, to ensure they adhere to safety and quality boundaries [2].

However, a primary challenge in implementing such benchmarking is the lack of standardized telemetry. While many agentic frameworks include native observability tools, each utilizes a unique telemetry model. This lack of uniformity hinders independent evaluation and complicates cross-framework comparison. To address this fragmentation, recent efforts have focused on defining semantic conventions for reporting telemetry data. Notably, the GenAI Observability initiative within OpenTelemetry [8] aims to establish shared semantic standards for AI-agent observability. Such conventions would enable more consistent performance measurement and facilitate rigorous comparisons across diverse agent frameworks.

The reviewed work shows how NFRs have evolved from traditional software quality models to frameworks tailored for ML systems, reflecting new characteristics such as data suitability, fairness, and explainability. It also highlights that practitioner perspectives align with these expanded needs and that NFR importance varies by domain. Finally, emerging research on agentic AI points to efforts to define observability and safety practices, along with current limitations in telemetry standardization that affect comparison across frameworks.

III. Methodology

We first start by clearly defining an NFR taxonomy to chart the scope of our analysis of NFR evaluation of AI Agents.

A. Creating an NFR Taxonomy

Existing NFR frameworks such as ISO/IEC 25010 [4], ISO/IEC 5338 [6], and the AI-focused ISO/IEC 25059 [5] provide well established quality models for traditional software and for machine learning systems. While models are valuable foundations, they do not fully account for the operational characteristics of AI agents. Recent work [3] on ML system quality highlights important dimensions such as robustness, data quality, fairness, and model reliability. While these dimensions remain relevant, they do not cover the full range of behaviors that emerge in agentic workflows. Unlike static models, agents perform sequential reasoning, introducing additional layers of complexity such as variability in workflow paths, dependency on tool calls, management of context windows, and emergent reasoning artifacts - introducing distinct failure modes. These characteristics fall outside the scope of existing taxonomies, which means that classical NFR categories must be adapted and extended to reflect the realities of agent based execution.

To close this gap, we propose an agent-first two level NFR taxonomy. Level 1 defines broad quality categories that remain aligned with established software engineering standards. Level 2 defines detailed, agent specific metrics that reflect concrete behaviors observed during planning, reasoning, and execution. This structure preserves compatibility with existing quality models while enabling more granular and operationally meaningful evaluation of agentic capabilities in production environments.

Table I summarizes the taxonomy. It outlines each Level 1 category, the associated Level 2 metrics, and the operational definitions required for practical measurement and assessment.

B. Measurement Methodologies

Table II summarizes the metrics implemented in this work. We tailor these metrics within the framework of IBM ITBench.

1) Langfuse Tracing of Agents: ITBench SRE and CISO Agents are built using the Crew AI framework. To capture the steps taken by the Crew AI agents, the agents are instrumented with Langfuse to capture comprehensive traces of the agent execution. To capture all orchestration flows and raw LLM

TABLE I
AI AGENT NFR TAXONOMY (LEVEL 1 & LEVEL 2 ATTRIBUTES)

Level 1 Category	Level 2 Metric	Definition
Performance	End-to-end latency	Total time from task initiation to completion.
	Planning overhead	Time spent in reasoning cycles, reflection loops, or plan construction.
	Tool call latency	Delay between issuing a tool request and receiving a response.
	Time-to-first-token	Time until the first token is generated after the request is sent.
	Throughput under load	Number of tasks the agent can process concurrently without degradation.
	Workflow path efficiency	Degree to which the agent follows minimal, intended steps (avoids detours).
	Context window utilization	How effectively the agent uses available context to avoid truncation or loss.
Cost Efficiency	Token consumption per task	Average number of tokens used to complete a task.
	API call count	Total number of tool/LLM calls invoked per workflow.
	Cost per successful/failed task	Monetary cost associated with completed and failed attempts.
	Retry-induced overhead	Additional cost caused by agent retries or error recovery loops.
	Prompt Completion Ratio	The ratio of input tokens consumed to output tokens generated.
Reliability	Task success rate	Percentage of tasks completed correctly.
	Error rate by type	Frequency of tool errors, reasoning errors, hallucinations, etc.
	Retry frequency	Number of retries required to complete a task.
	Idempotency	Whether repeated executions yield consistent, stable results.
	Regression resistance	Stability of behavior over time when fixes/updates are introduced.
Explainability	Trace completeness	Availability of plans, reasoning summaries, and tool I/O.
	Decision explainability	Clarity of why specific actions or tool calls were chosen.
	Workflow milestone visibility	Ability to observe intermediate stages of execution.
	Guardrail/policy adherence visibility	Whether the agent exposes safety/guardrail constraints in outputs.
	OpenTelemetry compatibility	Support for standardized tracing and observability frameworks.
Scalability	Concurrency capacity	Maximum parallel workflows without failure.
	Multi-agent coordination efficiency	Performance changes when multiple agents collaborate.
	Cross-domain scaling	Ability to maintain quality across different task domains.
Security	Prompt injection resilience	Ability to resist malicious or adversarial instructions.
	Access control adherence	Correct compliance with permission and role constraints.
	Data sanitization	Avoiding leakage of sensitive information to external models.
	Auditability	Ability to trace decisions, tool actions, and security-relevant events.
Usability	Human intervention frequency	How often humans must step in to correct or guide the agent.
	Time-to-correct	Time required for a human to correct or override the agent.
	Output interpretability	How easily humans understand the agent's outputs.

calls, additional instrumentors for Langchain, Crew AI and LiteLLM from the OpenInference library are added. Agent execution is captured in a single trace, the root span, by wrapping the core Crew AI `kickoff` function with Langfuse. This ensures all steps taken by the AI Agent appears as part of a single trace. Once the Agent completes, the trace is flushed to the Langfuse dashboard. The trace details are fetched programatically using the Langfuse API and stored in a JSON file for offline analysis. Each trace contains a list of obervations that represents each step taken by the Agent. Each observation details the specific parameters of that step like name, type, latency and token usages. These data points serve as the basis for calculating the NFRs listed in Table II.

2) Instrumenting Mini-SWE Agent: Mini-SWE Agent [13] is an AI Agent developed for solving Github issues, achieving scores greater than 74% on SWE-Bench Verified [12]. To calculate NFRs for the Mini-SWE Agent, modifications were implemtented to the Agent's core classes. Mini-SWE Agent's `LitellmModel` Class was augmented to capture granual token usage metrics, particularly, input, output and reasoning token counts, from the LLM's response. The `DefaultAgent` Class was extended to capture start and end times for terminal command executions and error logs. Using these captured metrics, NFRs for the Mini-SWE Agent were calculated.

3) Using vLLM for NFRs measurement: CrewAI and Langfuse do not provide granular system level metrics to calcualte NFRs crucial to understanding the behavior of systems under agentic workloads. This necessitated creation of a sandbox environment by locally hosting an LLM using vLLM framework. We automate the existing dockerized evaluation framework for ITBench CISO Agents. Table II summarizes the metrics obtained from the vLLM backend. KV-Cache and context win-

dow utilization are crucial metrics to measure the memory load the agentic framework creates on the system. Furthermore, we determine the prefix cache hit rate per task and per run to measure the diversity of paths the agent explores during execution. While a high hit rate is desirable for efficiency, it also indicates high repetition in agent execution steps, leading to redundant computation which could be avoided by agent architectural modification and tool orchestration. The following metrics were collected during evaluations

1) Max KV Cache Utilization: Gives insight into memory stress of the system under agentic workload
2) Prefix Cache Hit Rate: Measure context re-use via caching. While a high hit rate is desirable for inference speed, it is indicative of possible repetition in agent execution steps.
3) Average Time to First Token (TTFT): We obtain TTFT from vLLM directly without getting using CrewAI streaming mode (which includes network overhead).
4) Average Time Per Output Token (TPOT): Measures inter-token delay during generation.
5) Average Prefill & Decode Times: Provides granular insight into prefill and decode stages during LLM calls.

Table II provides the formulae used to calculate these metrics.

4) Streaming API Implementation for CISO Agents: The original ITBench framework does not natively support streaming APIs for agent execution. To capture fine-grained latency metrics critical for production deployment, specifically Time-to-First-Token (TTFT) and Token Generation Speed (TGS), we extended only the CISO agents to support streaming mode while leaving the SRE and Mini-SWE agents unchanged.

We modified the CISO agent tool implementations to enable streaming responses from the underlying `gpt-4o-mini` model, which is accessed via a remote API endpoint. The implementation intercepts LLM completion calls and attaches asynchronous token streaming handlers that record timestamps as tokens arrive.

For each streaming call, we record two key timestamps: (1) the time at which the request is sent, and (2) the time at which the first token is received. TTFT measures the elapsed time between these two events, capturing the prefill latency before token generation begins. Token Generation Speed (TGS), measured in tokens per second, quantifies the throughput during the decoding phase after the first token arrives. These streaming-specific metrics are integrated into the existing Langfuse traces for the CISO agents and complement the non-streaming metrics collected for SRE and Mini-SWE agents. Table II provides the precise formulas for calculating these metrics.

IV. Experiments

A. Comparing ReAct and Plan&Execute Architectures

We run evaluations on ITBench benchmark scenarios for both ReAct and Plan&Execute architectures for each Agent (except mini-SWE which supports only ReAct) to capture NFR metrics and to develop a comprehensive comparision of the strengths and weaknesses of each.

B. SRE Agents

The ITBench SRE Agent was tested using the Gemini 2.5 Pro model across two different architecture variations. The first version used the standard ReAct execution framework in Crew AI. For the second version, the `planning` field in the Crew object was set to `True` to add a planning phase for the Agent before execution, following a Plan&Execute framework. The ITBench SRE Agent and Mini-SWE Agent were tested on all 15 open-source SRE scenarios from the ITBench Hub.

The SRE scenarios from ITBench use a Kubernetes environment. The Agent is provided access to `kubectl` commands to get access to logs, monitor the cluster and make fixes. The Agent consumes alerts from a pre-configured observability endpoint to identify and diagnose system anomalies. The Agent uses commands like `kubetcl get pods` and `kubectl logs` to diagnose the issue. The incidents involve issues like connectivity issues, perfomance bottlenecks and configuration errors.

The experiments were run on a `c2-standard-8` instance with 8 vCPUs and 32 GB memory. The scenarios were deployed on a kind cluster running locally.

Experimental results for 4 out of the 15 incidents are shown in Table III. We provide our evaluation based on results from all 15 incidents. See AppendixA for an exhaustive list.

C. CISO Agents

KubernetesKyverno, *KubernetesKubectlOPA* and *KubernetesKyvernoUpdate* agents were evaluated in both ReAct and Plan&Execute configurations using Gemini2.5 Pro and locally hosted Qwen LLMs.

1) Scenario Description: ITBench [7] have open-sourced 4 CISO agents and corresponding scenarios for benchmarking. We evaluate 3 out of the 4 agents in our work. *KubernetesKyverno* agent deploys a Kyverno policy to detect containers using the host network to limit the number of the containers sharing that namespace. It is a single-step, well defined task involving creation of a new policy from scratch. *KubernetesKubectlOPA* creates a bash script to collect K8s resources and an OPA policy to check compliance. Agent must understand data collection and policy authoring in different formats - involving multi-step evaluation. *KubernetesKyvernoUpdate* reviews and modifies existing Kyverno policies to prohibit root user, only allow trusted registry images. This is the hardest task as the agent cannot create new resources and can only modify existing ones. The scenario adds 2 new complex requirements by having stricter evaluation such as checks for unwanted changes, resource naming convention and integration complexity.

2) Experiments with Gemini-2.5-Pro: Using the Gemini2.5-Pro APIs, the three agents were benchmarked to determine NFR requirements. Metrics obtained from Langfuse and CrewAI traces were aggregated to calculate proposed NFRs. Refer to Table II for evaluation results.

3) vLLM deployed Qwen2.5-Instruct models: We used *Qwen2.5-14B-Instruct-AWQ* for running comprehensive NFR evaluations with system-level metrics. A sandbox framework

TABLE II
DEFINITIONS OF NON-FUNCTIONAL REQUIREMENTS (NFRS) AND THEIR FORMULAS

Category	NFR	Formula
Latency-Related Metrics	End-to-End Latency	`root_span.end_time - root_span.start_time`
	Tool Call Latency	`obs.latency where obs.type == ''TOOL''`
	Time To First Token (TTFT)	`obs.completion_start_time - obs.llm_call`
	Token Generation per Second	`(obs.completion_end_time - obs.completion_start_time) / output_tokens`
	Tool Round Trip Time	`obs.tool_end_time - obs.tool_start_time`
Token Usage and Resource-Related	Planning Overhead	$\sum$ `reasoning tokens` / $\sum$ `output tokens`
	LLM Call Volume	`COUNT(obs) (obs.model is not None)`
	Token Usage per Task	`AVG(usage_details['total']) grouped by task_id`
	Prompt Completion Ratio	`num input tokens / num output tokens`
	Context Window Utilization	`(total_tokens / context_window_size) * 100%`
Reliability and Efficiency Rate Metrics	Error Rate	`(Total Errors / Total Operations)` $\times$ `100%`
	Tool Reuse Rate	`COUNT(obs.name=''tool repeated usage'') / COUNT(obs.type=''TOOL'')`
Streaming-Specific Metrics	Time To First Token (TTFT)	$t_{\text{first\_token}} - t_{\text{request\_sent}}$
	Token Generation Speed (TGS)	$N_{\text{output\_tokens}}/(t_{\text{completion\_end}} - t_{\text{first\_token}})$
	TTFT Variance	$\text{std}(\text{TTFT}_{\text{all\_LLM\_calls}})$
	TGS Consistency	$(\text{TGS}_{\text{min}}/\text{TGS}_{\text{mean}}) \times 100\%$
vLLM System Metrics*	Max KV Cache Utilization	`range max vllm:kv_cache_usage_perc`
	Prefix Cache Hit Rate	`increase(vllm:prefix_cache_hits_total[{t}s])/ increase(vllm:prefix_cache_queries_total[{t}s])`
	Time To First Token	`rate(vllm:time_to_first_token_seconds_sum[{d}s]) / rate(vllm:time_to_first_token_seconds_count[{d}s])`
	Time Per Output Token	`rate(vllm:inter_token_latency_seconds_sum[{d}s]) / rate(vllm:inter_token_latency_seconds_count[{d}s])`
	Avg. Prefill Times	`rate(vllm:request_prefill_time_seconds_sum[{d}s]) / rate(vllm:request_prefill_time_seconds_count[{d}s])`
	Avg. Decode Times	`rate(vllm:request_decode_time_seconds_sum[{d}s]) / rate(vllm:request_decode_time_seconds_count[{d}s])`

* Obtained from Prometheus PromQL queries scraping vLLM metrics endpoint at an interval of 5s

was created for isolated instantiation and evaluation of CISO Agents. Along with obtaining NFRs from Langfuse and CrewAI, a promtheus instance scraping vLLM metrics was queried periodically for calculating system specific telemetry useful for estimating NFRs which capture measuring model and system level behavior. Refer to Table II for evaluation results.

4) Streaming API Evaluation for CISO Agents: To study the impact of streaming APIs on perceived responsiveness and token-level generation behavior, we enabled streaming only for the CISO agents. All streaming experiments were conducted using the `gpt-4o-mini` model exposed via a hosted API endpoint, while SRE and Mini-SWE agents continued to use non-streaming calls.

We evaluated streaming-capable CISO agents on three ITBench scenarios: KubernetesKyverno, KubernetesKubectlOPA, and KubernetesKyvernoUpdate. For each scenario, we ran multiple streaming-enabled executions, logging TTFT and TGS for every LLM call in the trace. These runs share the same task configuration and agent prompts as the non-streaming baselines, isolating the effect of streaming on latency-related NFRs at the LLM layer.

V. RESULTS AND DISCUSSION

A. SRE Agents

Mini-SWE Agent is typically the fastest across all incidents, approximately 2x faster compared to Plan&Execute version of ITBench SRE and 13x faster compared to ITBench using ReAct, on average. This superior performance is primarily because it does not rely on external tool calls to run terminal commands like the ITBench SRE Agent, avoiding the overhead of external tool calls. It also makes fewer LLM calls achieving completion quickly and more cost efficiently.

ITBench SRE Agent with ReAct framework often takes the longest time. The iterative nature of ReAct agents often leads to "agentic loops," where the model becomes trapped in repetitive reasoning and action cycles. This behaviour not only inflates execution time but also increases the number of LLM calls thereby increasing cost significantly.

The Plan&Execute configuration of the ITBench SRE Agent generally yields the highest Prompt-Completion Ratio.

TABLE III
NFR METRICS FOR SELECT SRE SCENARIOS USING GEMINI 2.5 PRO

Metric	SRE(P&E)	SRE(ReAct)	Mini-SWE
Incident 1			
Planning Overhead	80.71%	92.83%	90.40%
E2E Latency (s)	1135	13126	218
P/C Ratio	42.80	9.11	37.00
LLM Calls	11	21	10
Incident 16			
Planning Overhead	85.45%	82.75%	82.20%
E2E Latency (s)	362	401	161
P/C Ratio	9.775	0.96	4.31
LLM Calls	21	16	10
Incident 23			
Planning Overhead	83.33%	86.48%	69.31%
E2E Latency (s)	255	49	136
P/C Ratio	15.28	15.04	3.53
LLM Calls	11	6	4
Incident 30			
Planning Overhead	80.74%	87.95%	91.82%
E2E Latency (s)	304	793	283
P/C Ratio	6.14	2.19	81.46
LLM Calls	16	37	10
Incident 102			
Planning Overhead	81.92%	81.69%	67.69%
E2E Latency (s)	136	358	296
P/C Ratio	75.53	4.68	2.62
LLM Calls	3	14	7

P/C Ratio = Prompt Completion Ratio

TABLE IV
REACT VS PLAN&EXECUTE AGENTS FOR CISO

Metric	1.gen-kyverno R	1.gen-kyverno P	2.gen-kubectlopa R	2.gen-kubectlopa P	4.upd-kyverno R	4.upd-kyverno P
Gemini-2.5-Pro						
E2E Latency (s)	71.8	65.5	61.3	75.2	243.0	256.9
LLM Calls	8	9	7	8	28	28
Input Tok (K)	13.1	19.2	13.2	19.2	181.4	134.9
Output Tok (K)	6.5	5.3	5.2	7.4	9.3	18.5
Reasoning Tok (K)	3.2	2.0	2.1	4.2	0.6	6.0
Plan Overhead (%)	49.1	38.3	40.8	56.9	6.0	32.4
P/C Ratio	2.02	3.65	2.56	2.61	19.51	7.29
Tool Usages	3	5	3	3	18	13
Tool Err (%)	0	0	0	25	10	0
Throughput (t/s)	90.0	80.2	84.2	98.0	38.3	72.0
Avg LLM time (s)	8.9	7.2	8.7	9.3	8.6	9.0
Avg TRTT (s)	4.1	4.0	5.8	2.9	6.8	6.1
Qwen2.5-14B-Instruct-AWQ						
E2E Latency (s)	193.2	208.3	26.5	71.6	81.6	75.5
LLM Calls	34	34	7	10	11	7
Input Tok (K)	111.1	115.5	10.5	18.3	15.8	9.4
Output Tok (K)	4.8	5.2	0.6	1.8	2.1	1.9
Reasoning Tok$^+$	0	0	0	0	0	0
Plan Overhead (%)$^+$	0	0	0	0	0	0
P/C Ratio	22.90	22.04	16.27	10.17	7.53	4.88
Tool Usages	20	20	6	9	6	3
Tool Err (%)	0	0	25	40	0	0
Throughput (t/s)	25.1	25.2	24.4	25.1	25.7	25.5
Avg LLM time(s)	5.6	6.1	3.7	7.1	7.4	10.7
Avg TRTT (s)	4.9	4.5	0.9	1.0	5.9	7.9
KV-Cache Util %*	19.76	19.57	4.2	5.25	8.21	6.6
Prefix-Cache Hit %*	91.47	90.32	62.96	65.18	68.76	41.92

R=ReAct, P=Plan&Execute, *=Obtained from vLLM,
+: ITBench agent prompt templates did not enable reasoning in Qwen2.5-14

On average, the Plan&Execute agent maintains a Prompt-Completion ratio more than five times higher than that of the ReAct agent, with peak performance exceeding a 15-fold difference. This is largely due to the inclusion of the initial planning context in subsequent LLM prompts, which increases the input token length relative to the generated output. Plan&Execute Agent usually completes the task in a shorter duration than the ReAct agent as it does not get stuck in non-productive reasoning loops. Given that input tokens are significantly less expensive than completion tokens across most LLM provider APIs, the Plan&Execute approach emerges as a more cost-effective architectural alternative for production-ready agents.

B. CISO Agents

We evaluated three distinct CISO agent architectures using both ReAct and Plan&Execute (P&E) reasoning frameworks. The study spanned three large language models: the proprietary Gemini-2.5-Pro (via API) and the open-weights Qwen-2.5-14B-Instruct, hosted on a local vLLM instance.

1) Gemini-2.5-Pro Performance: Our analysis reveals a performance trade-off contingent on task complexity. On simpler benchmarks (Scenarios 1 and 2), the ReAct framework outperformed Plan&Execute, as the computational overhead of the initial planning phase in Plan&Execute outweighed its strategic benefits. However, on complex tasks requiring long-horizon reasoning, Plan&Execute demonstrated superior efficacy. Despite the initial planning cost, Plan&Execute eventually achieved higher system throughput and lower aggregate resource consumption-evidenced by reduced total token usage and fewer tool and LLM invocations-by eliminating the redundant reasoning steps observed in ReAct.

2) Local Inference Analysis (vLLM): Qwen-2.5-14B yielded 0% success rate across all scenarios, indicating that the reasoning capabilities of these smaller-parameter models are currently insufficient for the multi-step investigation required in complex CISO workflows. Benchmarking revealed counter-intuitive latency patterns indicative of reasoning failures. Specifically, agents addressing the *KubernetesKyverno* scenario exhibited 78% higher average latency compared to the nominally more complex *KubernetesKyvernoUpdate* scenario. This latency spike was accompanied by elevated tool usage, total token consumption, and near-saturation of the KV cache. These metrics collectively suggest that the models succumbed to hallucination loops, generating prolonged but non-

functional output rather than converging on a solution. Analysis of the prefix cache hit rate provides further insight into these failure modes. The failing runs generally exhibited high prefix cache hit rates, indicative of repetitive execution loops with little variation in reasoning. Conversely, Plan&Execute configurations in Scenarios 1 and 4 displayed significantly lower prefix cache hit rates. In this context, the lower hit rate is a positive indicator, reflecting the agent's ability to explore diverse execution paths and generate heterogeneous steps, rather than stagnating in recursive states.

3) Streaming API Performance Analysis: Table V reports streaming-specific metrics for the CISO agents using `gpt-4o-mini`. The SRE and Mini-SWE agents are not included here because they do not use streaming.

TABLE V
STREAMING METRICS FOR CISO AGENTS USING `GPT-4O-MINI`

Metric	1.gen-kyverno	2.gen-kubectlopa	4.upd-kyverno
E2E Latency (s)	20.68	24.67	54.72
TTFT Mean (s)	0.794	0.768	1.692
TTFT Max (s)	1.339	0.992	8.084
TGS Mean (tok/s)	59.36	64.54	68.42
TGS Min (tok/s)	50.97	50.77	60.74
LLM Calls	5	9	8

C. NFR interpretation of ReAct vs Plan&Execute

The following are our key takeaways from our study:

1) Overhead in Short-Horizon Tasks: Plan&Execute architectures incur a significant computational cost during the initialization phase due to comprehensive planning. Consequently, for short-horizon or low-complexity tasks, ReAct-based agents demonstrate superior efficiency, as the overhead of a dedicated planning stage outweighs its benefits relative to the rapid execution of the task.
2) Efficiency in Complex Environments: Conversely, in long-horizon scenarios, ReAct agents often exhibit increased latency and higher token consumption driven by redundant reasoning cycles and unguided exploration. The structured decomposition inherent to Plan&Execute agents mitigates these inefficiencies, yielding higher success rates and lower aggregate operational costs by reducing the frequency of iterative LLM calls.
3) Interpretation of Cache Metrics: We observe that Plan&Execute agents typically manifest a lower prefix cache hit rate compared to ReAct baselines. While maximizing cache hits is generally a performance objective, in this context, a lower rate is indicative of favorable behavior: it suggests the agent is processing distinct, heterogeneous sub-tasks rather than stagnating in repetitive cognitive loops with identical context.
4) Token Economy and Error Propagation: Although Plan&Execute agents exhibit a higher prompt-to-completion ratio-often perceived as a latency bottleneck-this characteristic contributes to superior total token economy. By recycling the generated plan as a static context for subsequent steps, the architecture minimizes the need for verbose intermediate reasoning. This upfront "planning overhead" effectively acts as a regularizer against error propagation (cascading errors), resulting in improved throughput and higher overall accuracy.
5) Efficiency: The ReAct and Plan&Execute frameworks exhibit distinct cost efficiency profiles for the ITBench SRE Agent. ReAct agents frequently incur higher operational costs because their iterative nature leads to agentic loops. These repetitive reasoning and action cycles significantly increase the total number of LLM calls. In contrast, the Plan&Execute configuration provides a more economical alternative by maintaining a high Prompt-to-Completion Ratio. This framework prioritizes the use of cheaper input tokens for extensive planning context and generates fewer expensive output tokens. By avoiding non-productive reasoning cycles, the Plan&Execute approach completes tasks more rapidly and with lower overall token expenditure.

D. Streaming APIs and CISO Agent Responsiveness

While SRE and Mini-SWE agents in our study rely exclusively on non-streaming LLM calls, we extended the CISO agents to support streaming mode using `gpt-4o-mini`. This extension reveals that traditional end-to-end latency metrics provide an incomplete picture of production responsiveness, particularly for interactive security workflows where analysts monitor agent progress in real time.

Our streaming experiments demonstrate that CISO agents exhibit significant variance in Time-to-First-Token (TTFT), ranging from 23 milliseconds to over 8 seconds across different scenarios. This variance is driven by three factors: (1) context length - longer prompts in Plan&Execute agents correlate with increased prefill overhead, (2) cold-start effects - initial LLM calls show elevated TTFT, and (3) scenario complexity - tasks requiring extensive policy context (e.g., Scenario 4: KyvernoUpdate) consistently exceed 1.5 seconds TTFT.

In contrast, Token Generation Speed (TGS) remains relatively stable once decoding begins, ranging from 59 to 72 tokens/second for `gpt-4o-mini`. This consistency indicates that perceived latency is dominated by the prefill phase rather than token generation, making TTFT the critical metric for user-facing agent responsiveness.

For production CISO deployments where security analysts await agent-generated policy recommendations or compliance reports, TTFT directly impacts operator confidence. Scenarios exceeding 5-second TTFT may be perceived as system hangs, potentially leading analysts to prematurely terminate or distrust agent outputs. The streaming traces also enable detection of failure modes invisible in aggregate metrics, such as generation stalls (sudden TGS drops indicating model uncertainty) or premature termination (abnormally short outputs suggesting tool failures).

Comparing streaming-capable CISO agents with non-streaming SRE baselines highlights an architectural trade-off:

streaming introduces 2-5% computational overhead for callback handling and incremental token processing, but provides immediate feedback that reduces perceived latency and enables fine-grained debugging through token-level telemetry.

VI. Gaps and Research Opportunities

Despite advancements in observability tools and the existence of general software quality standards and AI-specific extensions, significant gaps remain in the practical benchmarking of AI Agents. Consequently, existing approaches often fail to address the complex, non-deterministic nature of agentic workflows. This section identifies critical limitations in current approaches and outlines key directions for future research.

A. Standardization and Benchmarking Frameworks

A primary limitation is the lack of standardized NFR benchmarking frameworks. While benchmarks such as IT-Bench, AgentBench, and WebArena [14] provide domain-specific functional evaluations, they lack instrumentation for essential NFRs like latency profiling, token tracking, and error classification. This absence prevents the systematic evaluation of architectural trade-offs and optimization strategies. There is an urgent need to develop standardized telemetry collection protocols compatible with diverse frameworks (e.g., LangChain, AutoGPT). Future work should focus on establishing baseline NFR profiles for common architectures and creating community-driven reference datasets with ground-truth NFR annotations. This would enable cross-framework comparisons and reproducible research without exposing proprietary enterprise information.

B. Adaptive Optimization and Generalization

Manual NFR optimization, such as static adjustments to context windows or retry policies, does not generalize well across diverse tasks or varying deployment environments. There is a significant opportunity for auto-tuning frameworks that leverage production telemetry to learn optimal configurations dynamically. Reinforcement learning approaches could optimize agent policies directly for NFR objectives, balancing task success with latency and cost constraints. Furthermore, hierarchical scheduling systems could be developed to route tasks to agent instances with appropriate resource allocations based on real-time system states (e.g., differentiating between peak and off-peak traffic requirements).

C. Unified NFR Scoring and Production-Readiness Score

While this work defines and measures a comprehensive set of non-functional requirements across performance, cost, reliability, and explainability, the current evaluation reports these metrics independently. A key remaining gap is the absence of a unified scoring mechanism that aggregates heterogeneous NFRs into a single production-readiness signal.

NFRs differ in scale, units, and domain importance, making naive aggregation inappropriate. For example, latency may dominate SRE workflows, while auditability and trace completeness are critical in CISO scenarios, motivating the need for domain-aware weighting. Future work should explore composite NFR scoring models that normalize and weight metrics based on operational priorities. A unified production-readiness score would enable clearer comparison across agent architectures and configurations while remaining interpretable through its underlying NFR components.

VII. Conclusion

A. Summary of Findings

AI agents are rapidly transitioning from research prototypes to operational components of IT automation pipelines, yet their evaluation remains dominated by functional success metrics that fail to capture the complexities of real-world deployment. This project demonstrates that existing software and AI quality frameworks, while comprehensive for traditional systems, do not fully account for the nondeterministic reasoning, dynamic tool use, and workflow variability that define agentic behavior. To bridge this gap, we introduced an agent-centric NFR taxonomy that extends established standards with fine-grained metrics reflective of planning quality, tool-call dynamics, context management, observability, and safety.

Our analysis of current benchmarks reveals that functional metrics alone are insufficient indicators of production readiness. We extend the ITBench SRE and CISO Agents to capture fine-grained NFRs. Benchmarking these agents gives us signifcant insight into the current state of NFRs for IT Automation Agents.

B. Contributions

- Arjun: Survey paper, looked at the integration of NFRs into ITBench, and evaluated Mini-SWE agent
- Ayush: Survey paper, instrumented CISO, SRE and mini-SWE Agents for NFRs, conducted experiments on SRE and mini-SWE Agents
- Gautam: Survey paper, implemented streaming API support for ITBench CISO agents using `gpt-4o-mini`, conducted streaming NFR experiments, and added streaming performance metrics to Langfuse-based traces.
- Rishabh: Literature review, maintained codebase and agent environment, vLLM hosting and benchmarking, CISO agent sandbox benchmarking implementation and evaluation, NFR analysis
- Tanmay: Literature review, installation for ITBench SRE agent environment, evaluations for CISO agent reliability

VIII. Acknowledgments

We would like to acknowledge that this work is the result of a collaborative team research effort. We extend our sincere gratitude to our supervisor, Ruchi Mahindru, Distinguished Engineer at IBM for her guidance, feedback, and support throughout the development of this paper. We also thank the supporting resources, tools, and academic infrastructure that facilitated our research and analysis.

REFERENCES

[1] Vincenzo De Martino and Fabio Palomba. Classification and challenges of non-functional requirements in ml-enabled systems: A systematic literature review. *Information and Software Technology*, 181:107678, 2025.

[2] Liming Dong, Qinghua Lu, and Liming Zhu. Agentops: Enabling observability of llm agents, 2024.

[3] Khan Mohammad Habibullah, Gregory Gay, and Jennifer Horkoff. Non-functional requirements for machine learning: understanding current use and challenges among practitioners. *Requirements Engineering*, 28(2):283–316, 2023.

[4] International Organization for Standardization. *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO/IEC, Geneva, Switzerland, 2011.

[5] International Organization for Standardization. *ISO/IEC 25059:2023 Software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model for AI systems*. ISO/IEC, Geneva, Switzerland, 2023.

[6] International Organization for Standardization. *ISO/IEC 5338:2023 Information technology — Artificial intelligence — AI system life cycle processes*. ISO/IEC, Geneva, Switzerland, 2023.

[7] Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, Noah Zheutlin, Saki Takano, Divya Pathak, Felix George, Xinbo Wu, Bekir O. Turkkan, Gerard Vanloo, Michael Nidd, Ting Dai, Oishik Chatterjee, Pranjal Gupta, Suranjana Samanta, Pooja Aggarwal, Rong Lee, Pavankumar Murali, Jae wook Ahn, Debanjana Kar, Ameet Rahane, Carlos Fonseca, Amit Paradkar, Yu Deng, Pratibha Moogi, Prateeti Mohapatra, Naoki Abe, Chandrasekhar Narayanaswami, Tianyin Xu, Lav R. Varshney, Ruchi Mahindru, Anca Sailer, Laura Shwartz, Daby Sow, Nicholas C. M. Fuller, and Ruchir Puri. Itbench: Evaluating ai agents across diverse real-world it automation tasks, 2025.

[8] Guangya Liu and Sujay Solomon. AI Agent Observability - Evolving Standards and Best Practices. OpenTelemetry Blog, March 2025.

[9] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2025.

[10] Dewi Mairiza, Didar Zowghi, and Nurie Nurmuliani. An investigation into the notion of non-functional requirements. In *Proceedings of the 2010 ACM Symposium on Applied Computing*, SAC '10, page 311–317, New York, NY, USA, 2010. Association for Computing Machinery.

[11] Koji Nakamichi, Kyoko Ohashi, Isao Namba, Rieko Yamamoto, Mikio Aoyama, Lisa Joeckel, Julien Siebert, and Jens Heidrich. Requirements-driven method to determine quality characteristics and measurements for machine learning software and its evaluation. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, pages 260–270, 2020.

[12] OpenAI. Introducing SWE-bench Verified. OpenAI Blog, August 2024.

[13] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[14] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024.

APPENDIX

Table VI shows the various NFRs calculated for the IT-Bench SRE Agents across all scenarios.

TABLE VI
NFR METRICS FOR ALL SRE SCENARIOS USING GEMINI 2.5 PRO

	Plan & Execute						ReAct					
Inc.	**PO%**	**E2E(s)**	**P/C**	**LLM**	**TRTT**	**Exec**	**PO%**	**E2E(s)**	**P/C**	**LLM**	**TRTT**	**Exec**
1	80.7	1135	42.8	11	4.0	7.8	92.8	13126	9.1	21	2.4	5.5
3	78.2	307	12.7	12	4.6	8.7	85.4	126	15.8	9	6.1	6.9
16	85.4	362	9.8	21	6.4	8.0	82.8	401	1.0	16	4.8	6.2
20	87.8	661	1.2	20	6.8	6.8	87.8	214	11.5	10	6.4	7.8
23	83.3	255	15.3	11	3.5	8.4	86.5	153	15.0	6	3.7	9.7
26	78.7	533	0.9	9	7.7	6.3	84.9	504	38.1	5	4.8	9.8
27	85.7	251	16.4	13	3.2	7.0	83.7	589	40.3	3	4.4	8.7
30	80.7	304	6.1	16	9.8	11.6	88.0	793	2.2	37	12.6	14.7
31	83.4	1064	13.3	19	8.2	8.3	86.5	218	9.5	12	5.4	9.0
33	73.5	925	16.8	7	8.4	7.4	90.5	1340	11.5	10	4.9	6.4
34	87.6	1446	14.2	21	5.0	6.3	89.3	655	2.6	41	8.5	9.7
37	89.2	622	1.0	16	7.1	7.5	88.6	433	10.9	15	5.0	8.4
38	76.9	609	32.5	8	5.9	9.5	80.2	923	13.5	8	7.1	6.7
102	81.9	136	75.5	3	0.0	8.6	81.7	475	4.7	14	10.7	13.3
105	86.7	179	19.1	9	6.9	8.1	90.0	311	11.5	13	4.9	6.4

PO = Planning Overhead, E2E = End to End Latency, P/C = Prompt to Completion Ratio, LLM = LLM Calls, TRTT = Avg Tool Latency(s), Exec = Avg LLM Execution Time(s)